

Introduction to Pandas Data Visualization using Matplotlib

Unit 5

What is Pandas?

- **Pandas is a Python library** used for data manipulation and analysis.
- **Provides two main data structures** – Series (1D) and DataFrame (2D).
- **Designed for structured data** like CSV, Excel, and SQL databases.
- **Supports data cleaning** – handling missing values, filtering, and aggregation.
- **Allows easy data transformation** like sorting, merging, and grouping.
- **Works well with NumPy and Matplotlib** for efficient data analysis and visualization.

Creating a Pandas Series

1. **Creates a Pandas Series** – A one-dimensional labeled array storing values [10, 20, 30, 40].
2. **Auto-indexing** – Each value gets an index starting from 0 (default).
3. **Prints the Series** – Displays values along with their index in a structured format.

Creating a Pandas Series

python

```
import pandas as pd  
data = pd.Series([10, 20, 30, 40])  
print(data)
```

- Output:

go

0 10

1 20

2 30

3 40

dtype: int64

Creating a Pandas DataFrame

1. **Creates a dictionary** with two keys: 'Name' and 'Age', storing lists of values.
2. **Converts the dictionary into a DataFrame** using `pd.DataFrame()`, organizing data in a tabular format.
3. **Prints the DataFrame**, displaying names and ages in a structured table with automatic indexing.

Creating a Pandas DataFrame

python

```
import pandas as pd
data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35]}
df = pd.DataFrame(data)
print(df)
```

- Output:

markdown

	Name	Age
0	Alice	25
1	Bob	30
2	Charlie	35

What is Matplotlib?

- **Matplotlib is a Python library** used for creating visualizations like graphs and charts.
- **Supports static, animated, and interactive plots** for data analysis.
- **Commonly used plots** include line plots, bar charts, scatter plots, histograms, and pie charts.
- **Highly customizable** – allows setting labels, titles, colors, and legends.
- **Works well with Pandas and NumPy** for easy data visualization.
- **Can save plots as images** in formats like PNG, JPG, and PDF.

Using Pandas with Matplotlib

Pandas integrates with Matplotlib for seamless visualization.

Workflow:

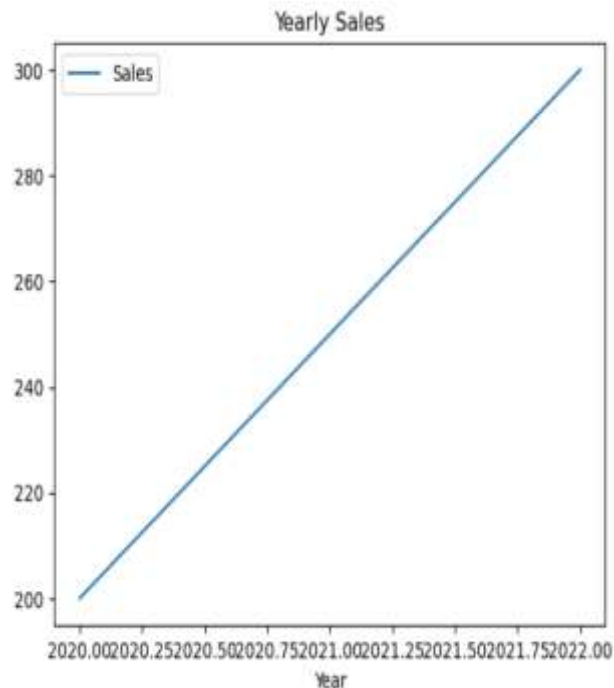
- Load data using Pandas.
- Perform analysis and transformations.
- Visualize using Matplotlib.

Using Pandas with Matplotlib

```
import pandas as pd
import matplotlib.pyplot as plt

# Load data
data = pd.DataFrame({'Year': [2020, 2021, 2022], 'Sales': [200, 250, 300]})

# Visualize
data.plot(x='Year', y='Sales', kind='line', title='Yearly Sales')
plt.show()
```



Reading Data from a CSV File

```
python
```

```
df = pd.read_csv('data.csv')  
print(df.head()) # Display first 5 rows
```

1. Reads a CSV file named 'data.csv' and loads it into a Pandas DataFrame.
2. `df.head()` displays the first 5 rows of the DataFrame for a quick preview of the data.
3. Helps in understanding the dataset structure, including columns and initial values.

Data Cleaning and Preprocessing

1. **Handling Missing Data** – `fillna(0)` replaces missing values with `0`, and `dropna()` removes rows with missing values.
2. **Filtering Data** – `df[df['Age'] > 30]` selects rows where the `Age` column has values greater than 30.
3. **Adding a New Column** – `df['Salary'] = [50000, 60000, 70000]` inserts a new column named "Salary" into the DataFrame.

Data Cleaning and Preprocessing

- **Handling missing values:**

```
python
```

```
df.fillna(0)  # Replace NaN with 0  
df.dropna()  # Remove rows with NaN values
```

- **Filtering data:**

```
python
```

```
df[df['Age'] > 30]
```

- **Adding new columns:**

```
python
```

```
df['Salary'] = [50000, 60000, 70000]
```

Basic Line Plot in Matplotlib

python

```
import matplotlib.pyplot as plt  
x = [1, 2, 3, 4, 5]  
y = [10, 20, 25, 30, 40]  
plt.plot(x, y)  
plt.xlabel('X-axis')  
plt.ylabel('Y-axis')  
plt.title('Line Plot Example')  
plt.show()
```

Basic Line Plot in Matplotlib

1. **Creates a line plot** using `plt.plot(x, y)`, where `x` represents the X-axis values and `y` represents the Y-axis values.
2. **Adds labels and a title** using `xlabel()`, `ylabel()`, and `title()` to describe the plot.
3. **Displays the plot** using `plt.show()`.

Bar Chart using Matplotlib

python

```
categories = ['A', 'B', 'C', 'D']  
values = [10, 20, 15, 25]  
plt.bar(categories, values, color='blue')  
plt.title('Bar Chart Example')  
plt.show()
```

Bar Chart using Matplotlib

1. **Creates a bar chart** using `plt.bar(categories, values)`, where categories `['A', 'B', 'C', 'D']` are on the X-axis and their values `[10, 20, 15, 25]` are on the Y-axis.
2. **Sets the bar color to blue** and adds a title using `plt.title()`.
3. **Displays the bar chart** using `plt.show()`.

Key Benefits

- **Pandas simplifies data manipulation** by providing efficient tools for handling structured data.
- **Matplotlib helps visualize data** through charts and graphs, making patterns easier to understand.
- **Together, they enable exploratory data analysis** by allowing users to clean, analyze, and visualize data seamlessly.
- **Supports various data formats** like CSV, Excel, and SQL for easy integration.
- **Helps in data-driven decision-making** by presenting insights clearly through graphs.
- **Widely used in data science and analytics** for tasks like trend analysis and pattern recognition.

The background features a minimalist geometric design. It includes several thin, light brown lines: a horizontal line near the top, a vertical line on the left, and a cross-like structure in the bottom right corner. There are also two small solid circles, one in the top right and one in the bottom left, both in a muted olive green color.

Thank You!

Unit-5

Handling exceptions using try-except else-finally.

1. Introduction
2. Try-Except in Python
3. How does the try-except work
4. Some examples
5. Else Block
6. Finally Keyword

1

try:



try to hit the ball

3

else:



else it will go in the hands of wicket keeper.

2

except:



If didn't hit,
then you will get
bowled.

4

finally:



Irrespective of
prior cases, the
ball will be thrown.

Introduction

- In python, an error can be of two types:
 - Syntax errors
 - Exceptions

Syntax errors: Errors are defined as the problems that cause the program to stop its execution.

Exceptions: Exceptions cause a change in the normal flow of the program due to some internal events that occurred in the program.

- Error in Python can be of two types i.e. **Syntax errors** and **Exceptions**.
- **Errors** are problems in a program, which will stop the program from execution.
- On the other hand, **exceptions** are raised when some internal events occur due to limitation of hardware or software part, which change the normal flow of the program.

Some common exceptions are:

- **SyntaxError:** This exception is raised when the interpreter encounters a **syntax error in the code**, such as a misspelled keyword, a missing colon, or an unbalanced parenthesis.
- **TypeError:** This exception is raised when an operation or function is applied to an object of the **wrong type**, such as adding a string to an integer.
- **NameError:** This exception is raised when a **variable or function name is not found in the current scope**.
- **IndexError:** This exception is raised when an **index is out of range** for a list, tuple, or other sequence types.

Some common exceptions are:

- **KeyError:** This exception is raised when a **key is not found** in a dictionary.
- **ValueError:** This exception is raised when a **function or method** is called with an invalid argument or input, such as trying to **convert a string to an integer** when the string does not represent a valid integer.
- **AttributeError:** This exception is raised when an **attribute or method is not found on an object**, such as trying to access a non-existent attribute of a class instance.
- **IOError:** This exception is raised when an I/O operation, such as **reading or writing a file**, fails due to an input/output error.
- **ZeroDivisionError:** This exception is raised when an attempt is made to divide a number by zero.
- **ImportError:** This exception is raised when an import statement fails to find or load a module.

Difference between Syntax Error and Exceptions

Syntax Error: This error is caused by the wrong syntax in the code.

```
if(amount > 999)
    print("amount more than 1000")
```

```
if(amount > 999)
```

```
SyntaxError: invalid syntax
```

Exceptions: Exceptions are raised when the program is syntactically correct, but the code results in an error. This error does not stop the execution of the program, however, it changes the normal flow of the program.

```
a = 10 / 0
```

```
print(a)
```

```
ZeroDivisionError: division by zero
```



Try-Except in Python

- In Python, to handle errors in a program, we use a try-except statement.
- **try block:** The try block is used when the code is needed to check for some errors. If there are no errors in the code then the code under the try block will get executed.
- **except block:** Whenever there is some error in the try block preceding the except block, the code under except block executes.

Syntax

```
try:
```

```
    # Some Code to try
```

```
except:
```

```
    # If an error is present in
```

```
    # the code try, then
```

```
    # code under except executes.
```

How does the try-except work

- First, the try block is executed.
- If any exception is not there in the code, then the try block will execute, except the block is finished.
- If an exception occurs then the except block will be executed and the try block is skipped.
- If an exception is still raised and the except block is left unhandled then it is passed to the outer try blocks. If still it is left unhandled then eventually the execution stops.
- A try statement can have more than one except block.

Some examples

- There is no exception, so try block will execute.

|

```
def divide(a, b):  
    try:  
        res = a // b  
        print("Answer: ", res)  
    except ZeroDivisionError:  
        print("Error dividing it by zero")  
  
divide(3, 2)
```

Output

```
Answer: 1
```

The try-except Block

What does it do?

- Tries to run a block of code.
- If an error occurs, it catches and handles it.

- **Example:**

```
try:  
    x = 10 / 0  
except ZeroDivisionError:  
    print("You can't divide by zero!")
```

- **Output:** You can't divide by zero!

- An exception is present, so the except block will run.

```
def divide(a, b):  
    try:  
        res = a // b  
        print("Answer: ", res)  
    except ZeroDivisionError:  
        print("Error dividing it by zero")  
  
divide(3, 0)
```

Output

```
Error dividing it by zero
```


Handling Multiple Errors

What if different errors can occur?

- Use multiple `except` blocks.
- **Example:**

```
try:  
    num = int("hello") # This will cause a ValueError  
except ValueError:  
    print("That's not a number!")  
except TypeError:  
    print("Wrong type of data!")
```

- **Output:** That's not a number!

Else Block

- An else block can also be added on the try-except block and it should be present **after all the except blocks**. If **the try block** successfully **does not raise an exception then** the code enters into the else block.

Syntax

```
try:
    # Some Code to check for errors
except:
    # if an error in the
    # try block then this block runs
else:
    # executes this block if
    # there is no exception in the try block
```

The else Clause

What is it for?

- Runs only if no error happens in `try` block.

- Example:

```
try:
    num = int("100")
except ValueError:
    print("Invalid input!")
else:
    print("Valid number:", num)
```

- Output: Valid number: 100

Code

```
def calc(x , y):  
    try:  
        z = ((x+y) // (x-y))  
    except ZeroDivisionError:  
        print ("The divide is resulting in 0")  
    else:  
        print (z)  
  
calc(2.0, 3.0)  
calc(3.0, 3.0)
```

Output

```
-5.0  
The divide is resulting in 0
```

Finally Keyword

- In Python, we use the finally keyword, which is executed always even if the exception is not handled and the try-except block is terminated. It is executed **after the try-except block**.

Syntax

```
try:
    # Some Code to check for errors
except:
    # If an error in the
    # try block then execute this block
else:
    # If no exception then execute this
finally:
    # This code always executed after the try-except
```

The finally Clause

What is it for?

- Runs no matter what, useful for cleanup tasks.
- Example:

```
try:  
    f = open("test.txt", "r")  
except FileNotFoundError:  
    print("File not found!")  
finally:  
    print("Execution completed.")
```

- Output:
 - If file is missing: File not found! then Execution completed.
 - If file is present: Execution completed.

Code

```
try:
    k = 5//0
    print(k)

except ZeroDivisionError:
    print("ZeroDivisionError")

finally:
    print("The code under finally is always executed")
```

Output

```
ZeroDivisionError
The code under finally is always executed
```

handling exceptions using try-except-finally blocks

try:

Some Code....which may have runtime error

except:

optional block

Handling of exception (if required)

else:

execute if no exception

finally:

Some code(always executed)



try:

```
k = 9//0 # raises divide by zero exception.  
print(k)
```

handles zerodivision exception

except ZeroDivisionError:

```
print("Can't divide by zero")
```

finally:

this block is always executed

regardless of exception generation.

```
print('This is always executed')
```